

Secure Software Design and Development
(CY-321)

Smashing the Stack For Fun And Profit

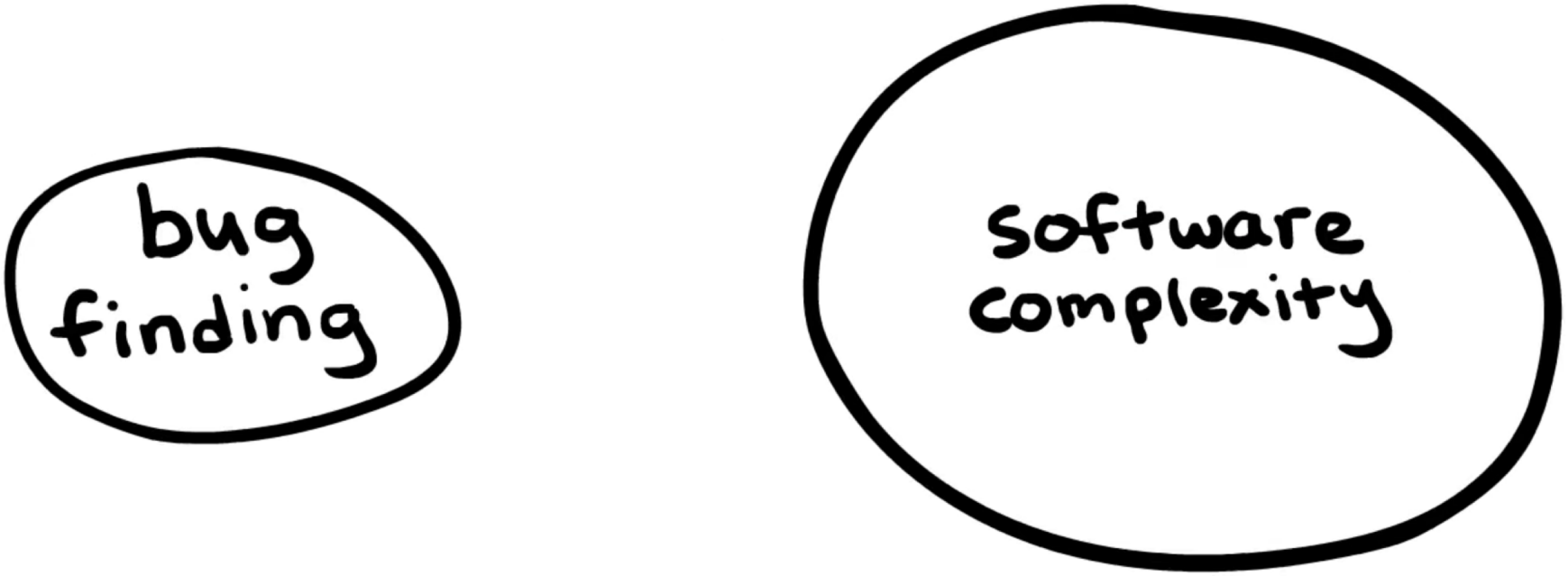
(Buffer Overflow)

Why Software Vulnerabilities Matter?

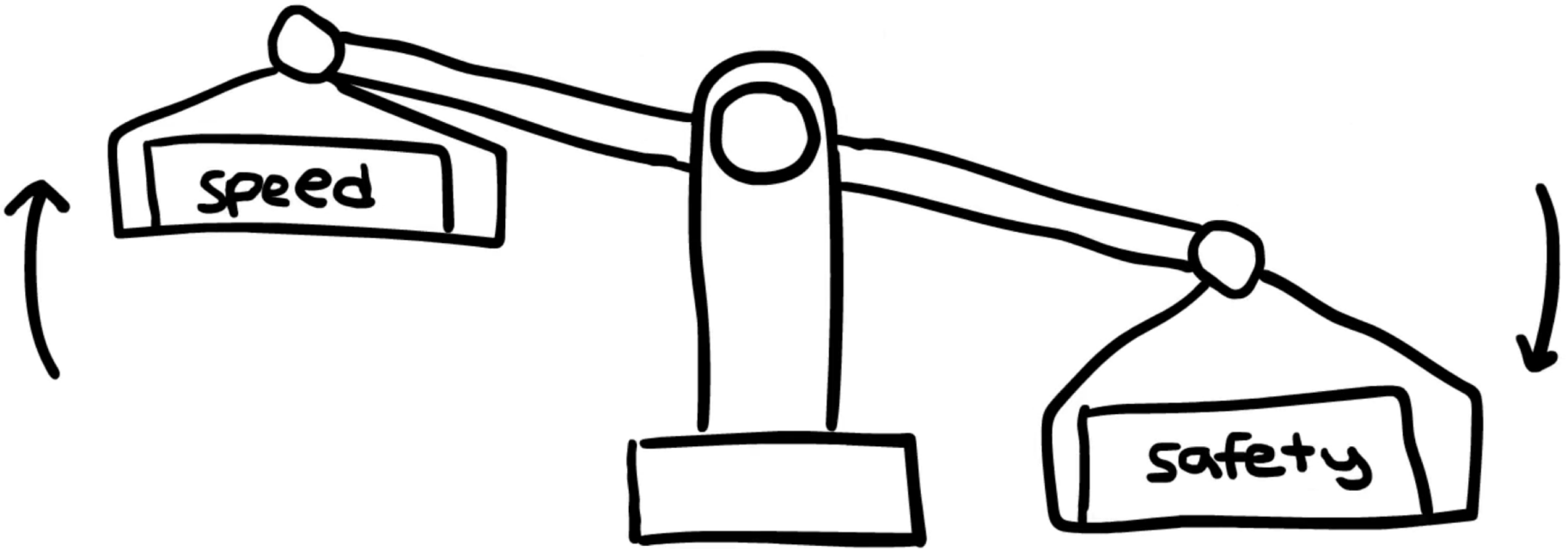
- When a process reads input from **attacker**, the process may be exploited if it contains vulnerabilities.
- When an **attacker** successfully exploits a vulnerability, he can
  - Crash programs: Compromises availability
  - Execute arbitrary code: Compromises integrity
  - Obtain sensitive information: Compromises confidentiality

- Software vulnerability enables the attacker to run with privileges of other users, violating desired access control policy

Buffer Overflow



Buffer Overflow



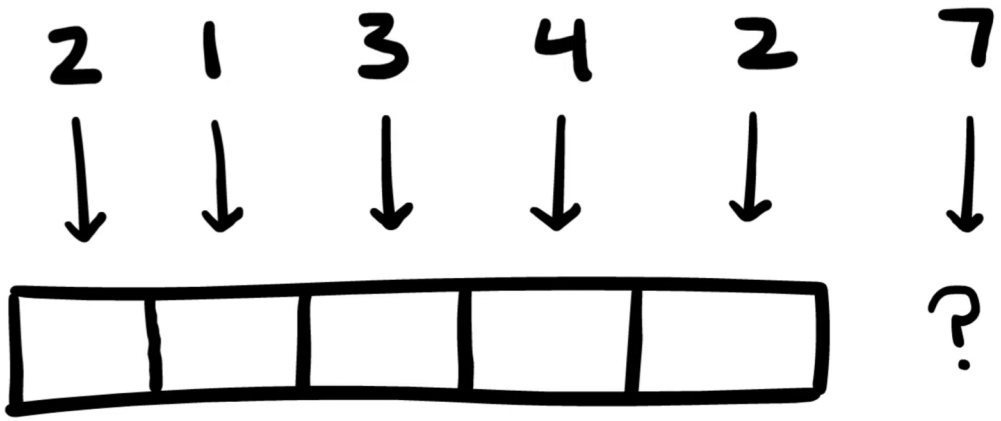
Attacks Exploiting Software Vulnerabilities

- Drive-by download (drive-by installation)
  - malicious web contents exploit vulnerabilities in browsers (or plugins) to download/install malware on victim system.
- Email attachments in PDF, Word, etc.
- Privilege escalation
  - Attacker on a system exploits vulnerability in a root process and gains root privilege

Buffer Overflow



Buffer Overflow



Buffers are most often used during I/O (reading or writing).

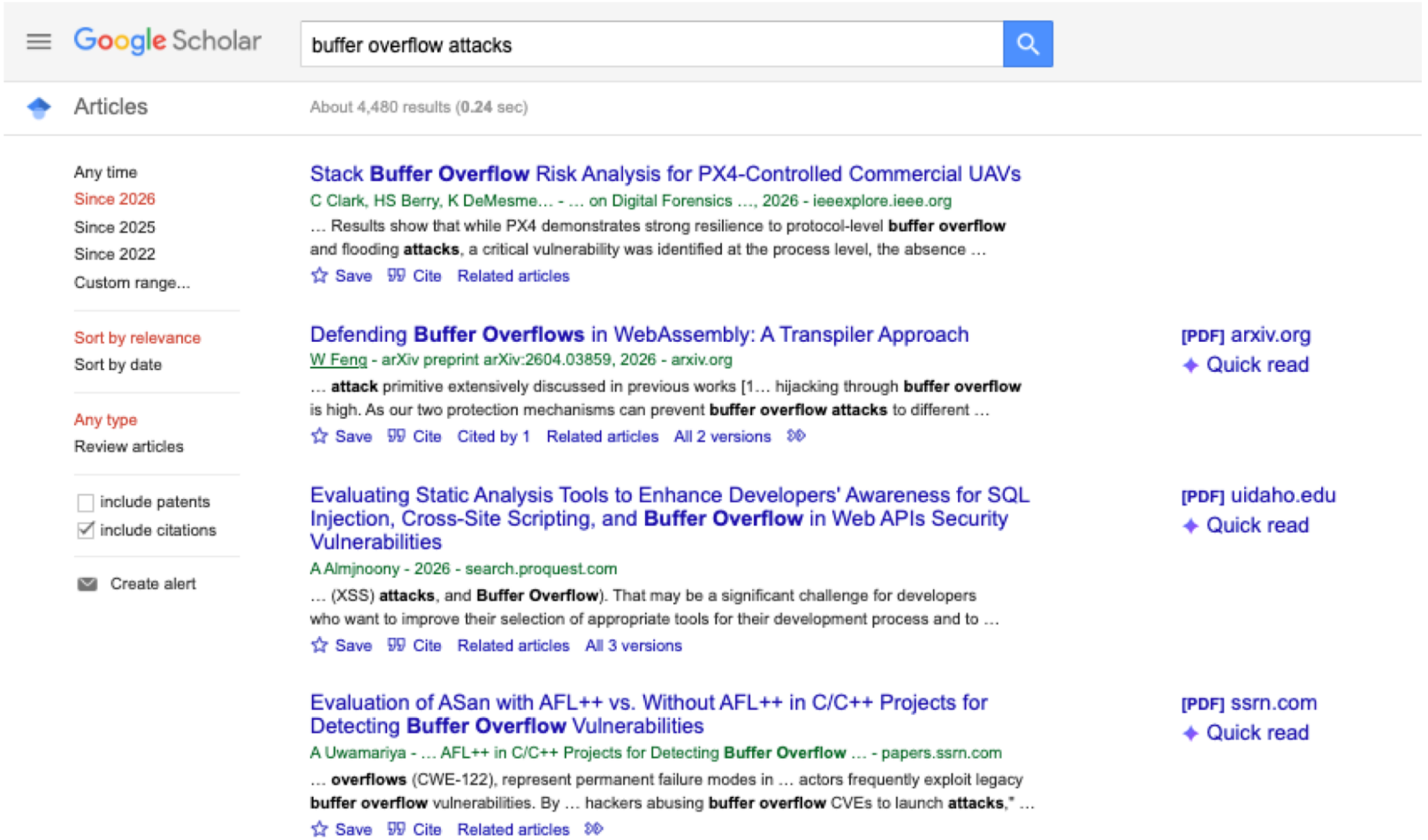
History of Buffer Overflow

Search Results

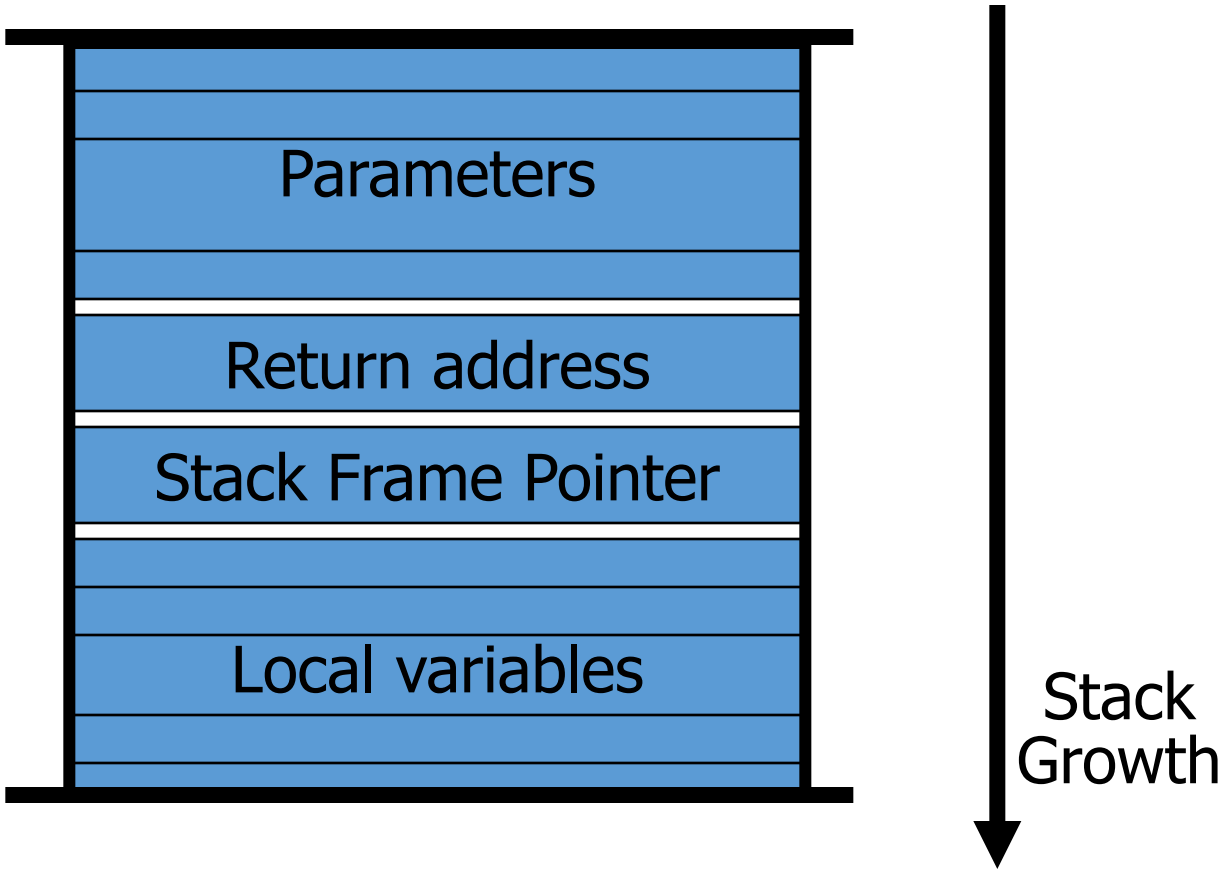
There are **11394** CVE Records that match your search.

Name	Description
CVE-2021-3410	A flaw was found in libcaca v0.99.beta19. A buffer overflow issue in caca_resize function in libcaca/caca/canvas.c may lead to local execution of arbitrary code in the user context.
CVE-2021-3382	Stack buffer overflow vulnerability in gitea 1.9.0 through 1.13.1 allows remote attackers to cause a denial of service (crash) via vectors related to a file path.
CVE-2021-3345	_gcry_md_block_write in cipher/hash-common.c in Libgcrypt version 1.9.0 has a heap-based buffer overflow when the digest final function sets a large count value. It is recommended to upgrade to 1.9.1 or later.
CVE-2021-3304	Sagemcom F@ST 3686 v2 3.495 devices have a buffer overflow via a long sessionKey to the goform/login URI.
CVE-2021-3182	** UNSUPPORTED WHEN ASSIGNED ** D-Link DCS-5220 devices have a buffer overflow. NOTE: This vulnerability only affects products that are no longer supported by the maintainer.
CVE-2021-3177	Python 3.x through 3.9.1 has a buffer overflow in PyCArg_repr in _ctypes/callproc.c, which may lead to remote code execution in certain Python applications that accept floating-point numbers as untrusted input, as demonstrated by a 1e300 argument to c_double.from_param. This occurs because sprintf is used unsafely.
CVE-2021-3156	Sudo before 1.9.5p2 has a Heap-based Buffer Overflow, allowing privilege escalation to root via "sudoedit -s" and a command-line argument that ends with a single backslash character.
CVE-2021-27232	The RTSPLive555.dll ActiveX control in Pelco Digital Sentry Server 7.18.72.11464 has a SetCameraConnectionParameter stack-based buffer overflow. This can be exploited by a remote attacker to potentially execute arbitrary attacker-supplied code. The victim would have to visit a malicious webpage using Internet Explorer where the exploit could be triggered.
CVE-2021-26825	An integer overflow issue exists in Godot Engine up to v3.2 that can be triggered when loading specially crafted.TGA image files. The vulnerability exists in ImageLoaderTGA::load_image() function at line: const size_t buffer_size = (tga_header.image_width * tga_header.image_height) * pixel_size; The bug leads to Dynamic stack buffer overflow. Depending on the context of the application, attack vector can be local or remote, and can lead to code execution and/or system crash.
CVE-2021-26713	A stack-based buffer overflow in res_rtp_asterisk.c in Sangoma Asterisk before 16.16.1, 17.x before 17.9.2, and 18.x before 18.2.1 and Certified Asterisk before 16.8-cert6 allows an authenticated WebRTC client to cause an Asterisk crash by sending multiple hold/unhold requests in quick succession. This is caused by a signedness comparison mismatch.
CVE-2021-26675	A stack-based buffer overflow in dnsproxy in ConnMan before 1.39 could be used by network adjacent attackers to execute code.
CVE-2021-26577	The Baseboard Management Controller (BMC) firmware in HPE Apollo 70 System prior to version 3.0.14.0 has a local buffer overflow in libifc.so uploadsshkey function.
CVE-2021-26573	The Baseboard Management Controller (BMC) firmware in HPE Apollo 70 System prior to version 3.0.14.0 has a local buffer overflow in libifc.so webgeneratesslcfg function.
CVE-2021-26572	The Baseboard Management Controller (BMC) firmware in HPE Apollo 70 System prior to version 3.0.14.0 has a local buffer overflow in libifc.so webgetactivexcfg function.

History of Buffer Overflow



Stack Frame


Parameters
Return address
Stack Frame Pointer
Local variables
Stack
Growth

Why Buffer Overflow happens?

```
void secret() {
    printf("Control Hijacked! We are inside secret()!
");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main");
    return 0;
}
```

Why Buffer Overflow happens?

```
void secret() {
    printf("\n Control Hijacked! We are inside
secret()!\n\n");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main.\n");
    return 0;
}
```

Why Buffer Overflow happens?

```
void secret() {
    printf("\n Control Hijacked! We are inside
secret()!\n\n");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main.\n");
    return 0;
}
```

Why Buffer Overflow happens?

```
void secret() {
    printf("\n Control Hijacked! We are inside
secret()!\n\n");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main.\n");
    return 0;
}
```

Why Buffer Overflow happens?

```
void secret() {
    printf("\n Control Hijacked! We are inside
secret()!\n\n");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main.\n");
    return 0;
}
```

Why Buffer Overflow happens?

```
void secret() {
    printf("\n Control Hijacked! We are inside
secret()!\n\n");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main.\n");
    return 0;
}
```

Why Buffer Overflow happens?

```
void secret() {
    printf("\n Control Hijacked! We are inside
secret()!\n\n");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main.\n");
    return 0;
}
```

Activation Frames Attacks

```
void secret() {
    printf("\n Control Hijacked! We are inside
secret()!\n\n");
}

void vulnerable() {
    char buffer[32];

    printf("Enter input: ");
    gets(buffer);
}

int main() {
    vulnerable();
    printf("Returned safely to main.\n");
    return 0;
}
```

Function Pointer Attacks

```
void safe_function() {
    printf("All good.\n");
}

void process(char *input) {
    char buffer[64];
    void (*fp)(void) = safe_function;

    strcpy(buffer, input
    fp();
}
```

longjmp Buffers

```
jmp_buf env;

void risky(char *input) {
    char overflowable_buf[64];

    strcpy(overflowable_buf, input);
    // ... later ...
    longjmp(env, 1);}

int main(int argc, char *argv[]) {
    if (setjmp(env) == 0) {
        risky(argv[1]);
    } else {
        printf("Back via longjmp.\n");
    }
}
```

Buffer Overflow

Code with Overflow

```
...
...

size_needed = (sizeof (*host_addr)+ sizeof (*h_addr_ptrs)
               + strlen (name) + 1);

host_addr = (host_addr_t *) *buffer;
h_addr_ptrs = (host_addr_list_t *)(( char  *) host_addr
               + sizeof (*host_addr));
hostname = (char *) h_addr_ptrs + sizeof (*h_addr_ptrs);
resbuf->h_name = strcpy (hostname, name);

...
...
```

Buffer Overflow

Code with Overflow

```
...
...

size_needed = (sizeof (*host_addr)+ sizeof (*h_addr_ptrs)
               + strlen (name) + 1);

host_addr = (host_addr_t *) *buffer;
h_addr_ptrs = (host_addr_list_t *)(( char  *) host_addr
               + sizeof (*host_addr));
hostname = (char *) h_addr_ptrs + sizeof (*h_addr_ptrs);
resbuf->h_name = strcpy (hostname, name);

...
...
```

Code w/o Overflow

```
...
...

size_needed = (sizeof (*host_addr) + sizeof (*h_addr_ptrs)
               + sizeof (*h_alias_ptr) + strlen (name) + 1);

host_addr = (host_addr_t *) *buffer;
h_addr_ptrs = (host_addr_list_t *)(( char  *) host_addr
               + sizeof (*host_addr));
hostname = (char *) h_addr_ptrs + sizeof (*h_addr_ptrs);
resbuf->h_name = strcpy (hostname, name);

...
...
```

fix

Why Buffer Overflow happens?

- Copying of data into the buffer without checking the size of input

```
#include <string.h>

void copy me(const char *s) {
    char copy[1024];
    strcpy(copy, s);
}
```

Why Buffer Overflow happens?

- Copying of data into the buffer without checking the size of input

```
#include <string.h>

void copy me(const char *s) {
    char copy[1024];
    strcpy(copy, s);
}
```

Why Buffer Overflow happens?

- Copying of data into the buffer without checking the size of input

```
#include <stdio.h>
#define BUFFER_SIZE 1024

void fill_buffer(char *buf, FILE *fp)
{
    fread(buf, 1, BUFFER SIZE, fp);
    if (!ferror(fp))
        buf[BUFFER SIZE] = '\0'; /*
Must null-terminate string */
}

10

void f() {
    char buf[BUFFER SIZE];
    fill buffer(buf, stdin);
}
```

Why Buffer Overflow happens?

- Accessing the buffer with incorrect length values

```
#include <stdio.h>
#include <string.h>

void unsafe_copy() {
    char buffer[10];
    printf("Enter a string: ");
    gets(buffer);
    printf("You entered: %s\n",
buffer);
}

int main() {
    unsafe_copy();
    return 0;
}
```

Why Buffer Overflow happens?

- Improper validation of array (simplest expression of a buffer) index

```
#include <stdio.h>

void unsafe_access() {
    int arr[5] = {1, 2, 3, 4, 5};
    int index;

    printf("Enter an index (0-4): ");
    scanf("%d", &index);

printf("Array element at index %d:
%d\n", index, arr[index]);
}

int main() {
    unsafe_access();
    return 0;
}
```

Why Buffer Overflow happens?

- Incorrect calculation of buffer size before its allocation

```
#include <stdio.h>
#include <string.h>

void unsafe_allocation() {
    char input[10];

    printf("Enter a string: ");
    fgets(input, 50, stdin);

    printf("You entered: %s\n",
input);
}

int main() {
    unsafe_allocation();
    return 0;
}
```

Why Buffer Overflow happens?

- Integer overflows or wraparounds

```
#include <stdio.h>

int main() {
    unsigned char x = 255;
    printf("Initial value of x: %u\n",
x);

    // Adding 1 will cause an overflow
    x = x + 1;

    printf("Value of x after overflow:
%u\n", x); return 0;
}
```

So Whats The Deal?

First, note that there is data as well as control flow information on the stack

If there is an automatic variable there that we can overflow, maybe we can overwrite the return address

If we can overwrite the return address, we can (usually) make the program execute code anywhere in the process address space

And if the variable we have overflowed is a buffer, why not fill the buffer with our code to execute?

This is what is meant **“allows execution of arbitrary code”**

Making an Exploit

```
#include <stdio.h>
#include <string.h>

void vulnerableFunction(char *input) {
    char buffer[16];
    strcpy(buffer, input);
    printf("Received: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        vulnerableFunction(argv[1]);
    } else {
        printf("Usage: %s <input>\n",
argv[0]);
    }

    return 0;
}
```

Explanation of the Vulnerability

The function **vulnerableFunction** defines a local buffer char `buffer[16];` that can hold up to 16 characters.

The function uses **strcpy** to copy the contents of the input (provided by the user via argv[1]) into the buffer. However, **strcpy** does not check the length of the input, so if the input is larger than 16 characters, it will overflow the buffer and overwrite adjacent memory on the stack.

```
#include <stdio.h>
#include <string.h>

void vulnerableFunction(char *input) {
    char buffer[16];
    strcpy(buffer, input);
    printf("Received: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        vulnerableFunction(argv[1]);
    } else {
        printf("Usage: %s <input>\n",
argv[0]);
    }

    return 0;
}
```

Potential Exploitation

When more than 16 characters are passed to the program as an argument, the **`strcpy`** function will copy the excess characters into memory locations beyond the buffer. This can overwrite the **return address** or other critical data on the stack, potentially allowing the attacker to:

- Execute arbitrary code (e.g., shellcode placed by the attacker).
- Redirect the program's execution flow to malicious code (e.g., jumping to the attacker's payload).

```
#include <stdio.h>
#include <string.h>

void vulnerableFunction(char *input) {
    char buffer[16];
    strcpy(buffer, input);
    printf("Received: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        vulnerableFunction(argv[1]);
    } else {
        printf("Usage: %s <input>\n",
argv[0]);
    }

    return 0;
}
```

Exploitation Example

```
./vulnerable_program AAAAAAAAAAAAAAAAAA
```

Here, the input string has 18 A characters, which exceeds the size of buffer (16 characters)

```
#include <stdio.h>
#include <string.h>

void vulnerableFunction(char *input) {
    char buffer[16];
    strcpy(buffer, input);
    printf("Received: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        vulnerableFunction(argv[1]);
    } else {
        printf("Usage: %s <input>\n",
argv[0]);
    }

    return 0;
}
```

Common Exploit Methods

Stack Smashing: By overflowing the buffer and overwriting the return address, the attacker can cause the program to jump to a location where they have injected malicious code (such as shellcode).

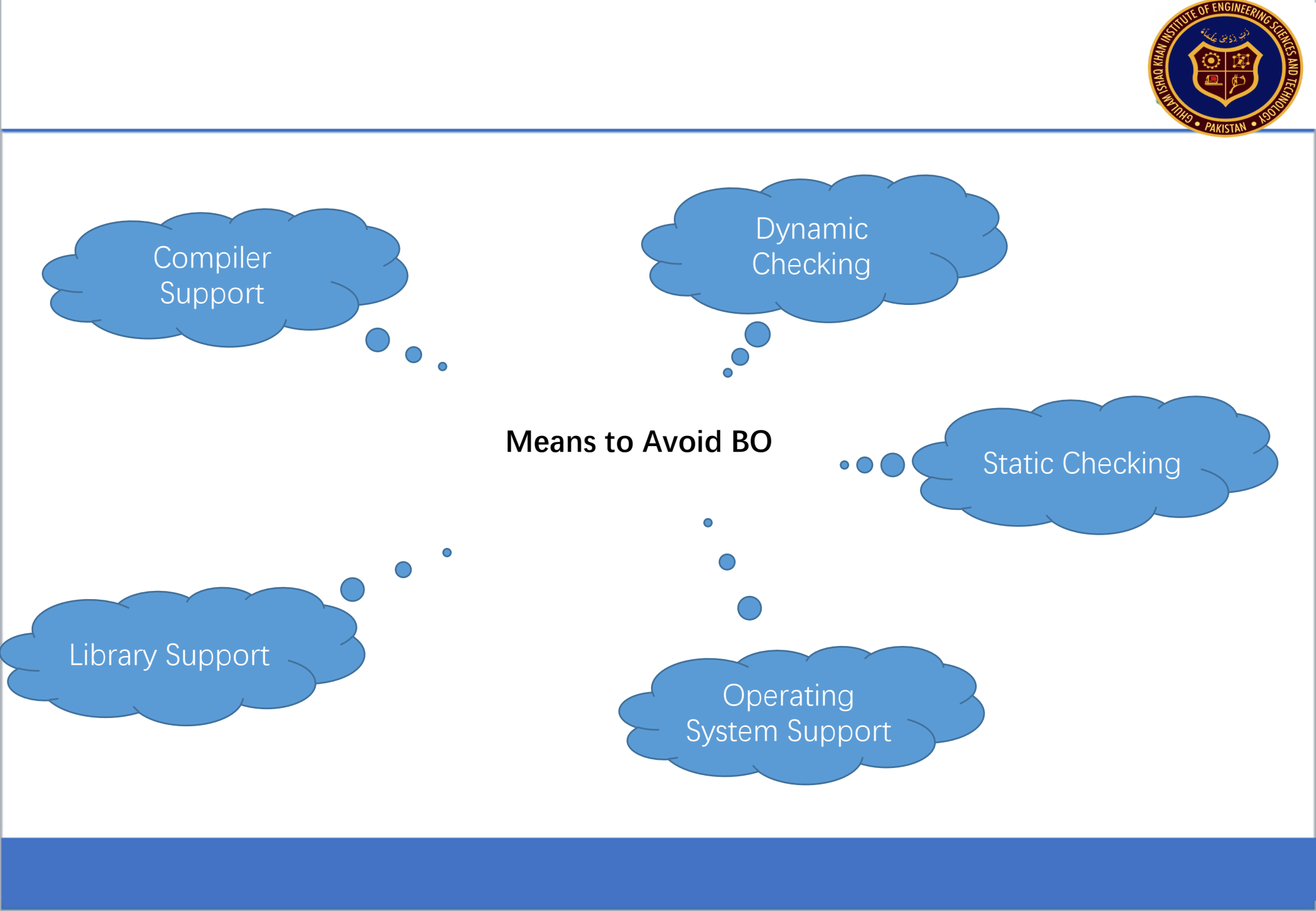
Return-to-libc Attacks: In the absence of executable stack protections (like DEP - Data Execution Prevention), an attacker might use a return-to-libc attack. Instead of injecting shellcode, they may overwrite the return address with the address of a function like system() in the C standard library to execute arbitrary commands

```
#include <stdio.h>
#include <string.h>

void vulnerableFunction(char *input) {
    char buffer[16];
    strcpy(buffer, input);
    printf("Received: %s\n", buffer);
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        vulnerableFunction(argv[1]);
    } else {
        printf("Usage: %s <input>\n",
argv[0]);
    }

    return 0;
}
```


Compiler Support
Dynamic Checking
Means to Avoid BO
Static Checking
Library Support
Operating System Support

Compiler Support

Products like **MemGuard** protect the return address on the stack by writing certain random values (**canaries**) on the stack in front of the return address and checking whether these values have changed just before returning.

Large performance impact

An attacker that can guess a canary value can attack the system

Operating System Support

ASLR (Address Space Layout Randomization): randomizes the location of stack, heap, and libraries, so attackers can't predict the addresses they need to jump to.

Library Support

Many overflows come from unsafe standard library functions like strcpy, gets, sprintf, and strcat, which don't check lengths.

Replace them with bounds-checked versions: strncpy, snprintf, fgets, strlcpy.

Use **safer library wrappers** (e.g., Libsafe) that intercept dangerous calls and ensure writes can't go past the stack frame.

Use safe string classes (C++ std::string) instead of raw char arrays.

Static Code Analysis

Some tools look at the source code to decide whether it is vulnerable.

Analyzing the code before it runs (at compile time or via tools) to find potential overflows.

Source code analyzers and linters (e.g., Coverity, Splint, Clang Static Analyzer).

Manual code review and auditing. Finds bugs early, but can produce false positives/negatives since it can't know actual runtime values.

Dynamic Methods

Work by checking every array and pointer access at run time

Since C has such an unwieldy array model, this has a huge performance impact

Also, the program cannot meaningfully continue to run after a buffer overflow has been detected ⇒ can be used only in development, not in production

Can only be used to find overflows as they occur (actual faults) as opposed to overflows that could occur (potential faults).

How to Write BO-Free Code



In practice, its hard to avoid them if they are at all possible

Its not enough simply to avoid certain functions (although that helps).

Its also not enough to make your source code open for peer review because most people simply dont do peer review:

> A buffer overflow was present for almost a decade in **wu-ftpd**, an FTP server program before it was finally noticed and removed.

Buffer overflows continue to be found in **sendmail**. Therefore:

Some Hard-And-Fast Rules

- Dont use C
- If you use C++, use **smart buffers** that do their own range checking
- Always include range checking code
- Never disable range checking code "for performance reasons"
- Use languages with managed memory like Java, Perl, Python
- Design your program so that it is secure from the start



Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby